

Intent-Driven Wireless Network Simulation and Optimization with Self-Evolving Agents

Anonymous Authors

Abstract—Wireless network simulation and optimization are difficult because they require solving a complex cross-layer configuration problem while interacting with complex 3D radio environments. For decades, engineers have relied on channel models, channel simulators, and lengthy standards to guide this process, but using them correctly still requires substantial domain expertise and engineering effort. Recent language-model agents can automate parts of wireless reasoning and code generation, but they remain preliminary: most operate on text, formulas, or single-layer tasks and do not construct 3D radio environments or run cross-layer simulations inside them. We present AGENTICSIONNA, a self-evolving language-agent system for intent-driven wireless network simulation. AGENTICSIONNA converts a natural-language request into an executable *radio world*: a 3D radio environment and a cross-layer wireless-system configuration and optimization. The system uses a multi-agent architecture with specialized scene-building, simulation, reflection, planning, and skill-learning agents. It learns from previous tutorials, worked examples, prior simulation results, and its own failed trajectories, then updates a procedural skill and knowledge base through verifier-driven optimization. We evaluate AGENTICSIONNA on benchmark suites covering radio-world generation and wireless simulation across multiple indoor and outdoor scenarios. On a customized wireless system simulation benchmark, the curated skill reaches 72.5% held-out pass rate versus 0.0% for both no-skill and self-generated-skill baselines. On wireless simulation tasks, AGENTICSIONNA achieves 92% execution success versus 22% for a raw LLM, with 89% physical plausibility.

I. INTRODUCTION

Wireless-network deployment is a high-stakes engineering task whose difficulty appears before any optimization or planning algorithm can be run. A deployment question must first be translated into a faithful cross-layer simulation problem: what physical environment is being modeled, what radio system is operating inside it, which radio-access, PHY, and system assumptions are active, and what evidence would make the result trustworthy. This translation is hard not merely because a wireless network is tied to the physical world, or because it has a large parameter space. It is a coupled network system: the same radio configuration can behave differently across an open office, a partitioned factory floor, or a multi-room building because propagation is shaped by geometry, radio materials, and device placement, while the meaning of that propagation depends on radio access choices, physical-layer settings, link behavior, and system-level objectives [1], [2].

The wireless community has spent decades managing this complexity through standardization and softwareization. Industrial and academic engineers document common assumptions in standards, channel models, and material recommendations, including 3GPP TR 38.901 and ITU-R P.2040 [3], [4]. They

also release software platforms that encode this expertise into reusable tools, ranging from packet-level network simulators and commercial planning suites to channel-sounding and radio-propagation toolchains [5]–[7]. In this paper, we use NVIDIA Sionna as the representative backend: Sionna is an open-source wireless simulation ecosystem that spans radio-environment modeling, link-level communication experiments, and system-level studies [8], [9]. Recent integrations further connect Sionna with network simulators and wireless digital-twin stacks [10], [11]. These efforts make wireless research reproducible, but they do not make it simple. As networks become denser and more environment-dependent, users must still read large bodies of documentation, choose compatible abstractions across layers, assemble simulator artifacts, and interpret multi-modal results. The expertise bottleneck therefore shifts from writing every model from scratch to keeping standardized knowledge, simulator software, physical assumptions, and network objectives mutually consistent.

The emergence of large language models has opened a new path for reducing this burden. LLMs can already generate code and operate tools in many software-engineering settings [12], [13], and recent wireless agents use LLMs for network slicing, mobile service assurance, wireless homework, radio-map generation, base-station siting, and optimization formulation [14]–[18]. These systems show that wireless tasks benefit from structured agent workflows, but they also expose a missing capability. If an LLM agent is asked only to operate a network through code or text interfaces, it can remain at the level of abstract states, logs, and optimization variables. A wireless simulation agent cannot: it must act more like an embodied agent living inside a radio world, where every network decision is tied to physical geometry, radio materials, device topology, protocol settings, and downstream objectives. This requirement is also different from existing language-guided 3D scene generation, which focuses on visually plausible layouts or embodied-AI environments [19]–[21]. For wireless simulation, the scene is not the endpoint; it is the substrate on which the radio system operates. The agent must understand which objects matter for propagation, how material choices affect the channel, how topology changes the wireless network, how physical-layer choices change the interpretation of the result, and how all of these choices should be translated into simulator artifacts and follow-up experiments. Existing wireless agents largely operate on abstract network states, optimization variables, or existing radio maps; they do not close this language-to-cross-layer-simulation understanding gap.

Our benchmark experiments (Section. xxx) confirm this gap

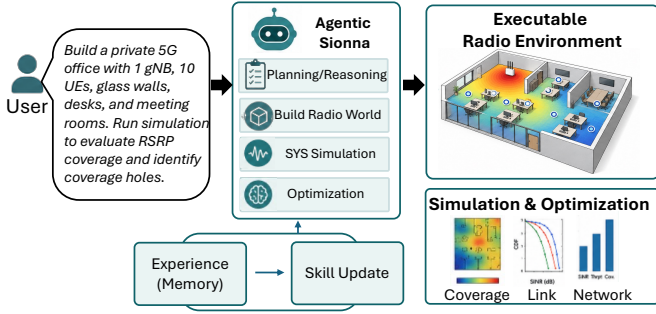


Fig. 1: **AGENTICSIONNA converts natural language into executable radio worlds.** A user describes a wireless simulation goal; the agent builds a 3D radio environment, binds it to network and experiment assumptions, executes Sionna simulations, reports multi-modal results, and updates its skill from failures.

by running a general coding agent in a coupled Sionna setting: when it is asked to turn a natural-language indoor scenario into both a 3D radio world and an executable wireless simulation, it produces executable code only **22%** of the time, and fewer than **10%** of outputs are physically meaningful. Training a new wireless agent from scratch is also impractical. Unlike general coding agents, which can draw on large software repositories, wireless simulation lacks a large, high-quality corpus that pairs tasks, simulator code, radio-world scenes, and verified outputs. Learning instead from engineer experience through human feedback, reinforcement learning, or execution traces is expensive because correctness often requires domain expertise, simulator execution, and inspection of the resulting radio map.

To bridge this gap, we present AGENTICSIONNA, a self-evolving LLM agent for wireless network simulation and planning, as shown in Fig. 1. AGENTICSIONNA treats wireless simulation as reasoning inside an editable *radio world*: it learns from prior materials and its own trials, understands 3D geometry and radio materials, creates and edits radio environments, runs cross-layer network simulations inside them, interprets multi-modal outputs, and uses the results to decide follow-up experiments. This makes AGENTICSIONNA different from a generic coding assistant: the agent is not only writing simulator code, but also maintaining the world, system, and experiment assumptions that make the code physically meaningful.

Realizing this vision requires solving three linked problems, each paired with a system mechanism:

First, operating a 3D radio world is difficult because existing coding agents do not have a reliable interface for understanding and manipulating coupled radio environments, where geometry, radio-material assignment, device placement, network topology, and simulator export must stay consistent. AGENTICSIONNA addresses this with a Sionna-based radio-world skill that exposes these operations as procedural tools: the agent edits a structured scene representation, applies object, material, and device-topology rules, validates layout constraints, and uses rendered scenes and radio maps as feedback.

Second, reasoning through wireless simulation tasks requires more than a general ReAct-style coding loop, because ordinary

software agents do not capture how wireless engineers carry assumptions from a scenario across radio access choices, physical-layer parameters, simulator code, metrics, and follow-up experiments. AGENTICSIONNA adapts this loop into a hierarchical learning process: it bootstraps from retrieved tutorials, repositories, and prior cases; runs and repairs simulations with execution feedback; and converts failure cases into auditable skill updates that can be reused on future tasks.

Finally, benchmarking radio-world competence is itself missing: there is no standard benchmark for measuring whether an agent can turn language into executable, physically meaningful wireless simulations. AGENTICSIONNA therefore builds a Sionna-centered benchmark in the spirit of execution-grounded software benchmarks: we distill tasks from public tutorials, documentation, and worked examples; stratify them by radio-world, link-level, and system-level difficulty; and evaluate them with verifiers that check both executable artifacts and cross-layer domain constraints.

This paper makes three contributions:

- 1) **Executable radio-world generation.** AGENTICSIONNA converts natural-language environment descriptions into simulation-ready radio worlds by constructing 3D geometry, assigning radio materials, placing devices, validating layout constraints, and exporting artifacts that a wireless simulator can load.
- 2) **Agentic wireless simulation.** AGENTICSIONNA turns open-ended user requests into executable Sionna experiments across environment-level, link-level, and system-level studies, while preserving cross-layer assumptions and using multi-modal diagnosis over heatmaps and metrics to drive follow-up experiments.
- 3) **Self-improving simulation skills.** AGENTICSIONNA improves a human-readable SKILL.md substrate through failure-driven discrete updates, evaluated with execution-based wireless verifiers rather than prompt inspection.

We instantiate AGENTICSIONNA on NVIDIA Sionna with Claude Sonnet 4.6 as the primary LLM backbone. On a 100-task indoor radio-world suite, `with_skill_v5` reaches **72.5%** held-out pass rate versus **0.0%** for no-skill and self-generated-skill baselines. Across single-shot wireless simulation tasks, AGENTICSIONNA achieves **92%** execution success versus **22%** for raw LLM and **35%** for RAG. In iterative radio-world experiments, it reaches **93.8%** average coverage in **3.4** iterations, within **2.4** percentage points of a grid-search upper bound while using far fewer simulations.

II. RELATED WORK

LLM agents for wireless networks. Recent work has begun to use LLM agents for wireless-network reasoning and control. WirelessAgent [14] introduces a structured workflow agent with perception, memory, planning, and action modules for network slicing, showing that tool-augmented workflows outperform prompt-only baselines and approach rule-based allocation. WirelessAgent++ [15] automates the design of such workflows by representing an agent workflow as executable

code over modular operators, optimizing it with domain-adapted Monte Carlo tree search, and introducing WirelessBench for wireless homework, network slicing, and mobile service assurance. ComAgent [18] coordinates multiple LLMs for solver-ready wireless optimization formulations, while LLM radio-map agents and accessible network-control systems target outdoor planning or configuration interfaces [16], [22]. These systems establish the value of structured wireless agents, but their tasks are network reasoning, slicing, service assurance, or formulation; they do not construct simulation-ready 3D radio environments, keep cross-layer simulation assumptions consistent, diagnose spatial results, or autonomously improve the simulation skill.

LLM-assisted wireless and engineering simulation. RadioSim Agent [23] is the closest prior system: it couples an LLM with deterministic radio simulators and vision-enabled radio-map interpretation. However, it focuses on interactive radio-map analysis over existing scenarios rather than natural-language radio-world construction, cross-layer communication experiments, or skill evolution. Concurrent work integrating LLMs with ns-3 targets packet-level simulation generation rather than 3D radio-environment simulation [24], and VLM-guided differentiable radio simulation estimates material parameters for a fixed scene, the inverse of our scene-construction problem [25]. Across adjacent engineering domains, FeaGPT [26], AutoMOOSE [27], MCP-SIM [28], and COSMO-Agent [29] validate agentic simulation pipelines in finite-element analysis, phase-field modeling, and general physics. AGENTICSIONNA contributes the wireless-specific machinery these systems lack: radio-world construction, executable cross-layer wireless-simulation workflows, spatial diagnosis, and simulation-skill improvement.

Skills, workflow optimization, and benchmarks. AGENTICSIONNA builds on the tool-augmented paradigm introduced by ReAct [30] and on recent work formalizing agent skills as portable procedural capabilities [31], [32]. Workflow-optimization systems such as AFlow [33] and WirelessAgent++ search over agent programs, while SKILLRL distills trajectories into reusable skills [34]. Our skill iteration has a different engineering target: it edits a human-readable SKILL.md substrate, not model weights or generic workflow code, and accepts edits only through execution-based wireless validation. This follows the philosophy of SWE-bench-style evaluation [35], but our verifiers are wireless-specific: scene geometry and radio-material checks, simulator loadability, executable wireless simulations, and coverage improvement across iterations.

3D scenes and wireless simulation backends. LLM-driven scene generators such as LayoutGPT, FlairGPT, Holodeck, and OptiScene produce visually plausible indoor environments [19]–[21], [36]. They do not, however, assign materials by radio behavior or validate whether the resulting scene can be executed by a wireless simulator. In parallel, Sionna and Sionna RT provide programmable radio-environment and physical-layer simulation [8], [9], with ecosystem integrations such as ns3-Sionna and Open Wireless Digital Twin [10], [11]. AGENTICSIONNA bridges these lines of work by making the

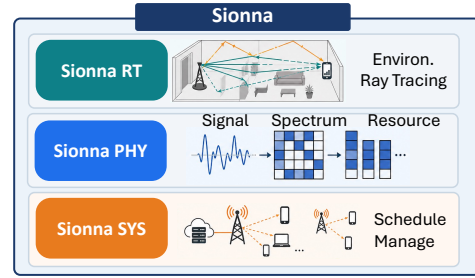


Fig. 2: **Sionna Wireless Network Simulator Architecture.** Sionna connects 3D radio environments, physical-layer experiments, and system-level studies.

generated scene simulation-ready before invoking downstream wireless experiments.

III. BACKGROUND: WIRELESS SIMULATION WITH SIONNA

Sionna as a simulation ecosystem. We use NVIDIA Sionna as the representative backend because it is a mature open-source ecosystem for wireless-system research rather than a single-purpose simulator [8], [37]. Its stack connects three levels that are usually handled by different tools. At the environment level, *Sionna RT* builds 3D radio scenes, models propagation through geometry and radio materials, and produces channels and radio maps [9]. At the communication level, *Sionna PHY* provides programmable physical-layer and link-level building blocks for experiments such as modulation, coding, MIMO processing, and error-rate evaluation. At the network-abstraction level, *Sionna SYS* supports system-level studies based on physical-layer abstraction. This layered design is what makes Sionna attractive for AGENTICSIONNA and also what makes it hard for agents: the same backend can start from a constructed radio world, run scene-dependent propagation, and continue into downstream link- or system-level experiments, but the assumptions across those layers must remain compatible.

From a radio world to a simulation artifact. A Sionna workflow is not just a code-generation problem. The user intent must be translated into a coupled cross-layer artifact that includes (1) a 3D radio environment with geometry, obstacles, and radio-material assignments, (2) a radio-system configuration with device topology, carrier settings, antennas, channels, and transmission choices, (3) an executable experiment such as coverage analysis, link-quality evaluation, MIMO comparison, or follow-up placement search, and (4) an interpretation of the resulting metrics, plots, and radio maps. These pieces are coupled: changing a wall material can change a radio map; changing device placement can change the link experiment; changing the physical-layer setup can change which propagation outputs are meaningful; and changing the system objective can change which simulator outputs matter. Human engineers manage these dependencies by reading documentation, adapting examples, running the simulator, and inspecting intermediate outputs.

Why this stack is hard for agents. Sionna exposes the right abstractions, but it also exposes the main difficulty for LLM agents: correctness is distributed across the whole wireless stack rather than localized in one API call. A valid simulation

requires consistency across scene geometry, material models, exported files, device placement, channel assumptions, physical-layer parameters, requested metrics, system objectives, and plotting or reporting code. Many failures are semantic rather than syntactic: a scene may load but use visually plausible materials with implausible radio behavior, a script may execute but evaluate a mismatched channel assumption, or a heatmap may look reasonable while answering the wrong deployment question. Fig. 2 summarizes this documentation-to-artifact burden and the points where AGENTICSIONNA inserts radio-world skills, Sionna worked examples, and execution-based checks.

IV. SYSTEM OVERVIEW

AGENTICSIONNA is a session-level agent that turns a user's wireless question into an editable radio world, an executable cross-layer Sionna experiment, an interpretation of the result, and a record of what the agent learned. Fig. 3 shows the architecture. The key design choice is to separate the workflow into specialized workers while keeping a single Orchestrator responsible for user intent, session state, and artifact consistency.

The Orchestrator first converts a natural-language request into a typed simulation plan. The plan records the scenario, required radio-world artifacts, target wireless experiment, cross-layer assumptions, constraints, expected outputs, and stopping condition. It then dispatches workers with scoped skills rather than asking one model context to handle 3D geometry, Sionna APIs, plotting, and diagnosis at the same time. All workers return typed artifacts, such as a scene state, a Sionna script, a result bundle, a diagnostic report, or a skill-edit proposal.

The Scene Builder owns the radio world. It creates or edits a structured scene representation, grounds objects in an asset catalog, assigns radio materials, validates geometry and clearance constraints, and exports the scene into visual and simulator-ready formats. The Simulator owns executable wireless experiments. It adapts Sionna worked examples to the current task, runs the generated code, and records the simulation parameters, plots, metrics, and provenance. The Reflector and Experiment Planner close the loop after execution: the Reflector explains heatmaps and numerical metrics in the context of the scene, while the Planner turns that diagnosis into follow-up simulations such as access-point relocation, carrier changes, or parameter sweeps. Finally, the Skill Learner turns repeated cross-layer failures into reusable procedural knowledge by editing the agent's human-readable skill rather than training a new model.

This architecture lets AGENTICSIONNA behave as a radio-world agent rather than a one-shot code generator. It can understand a prompt, build the world implied by the prompt, run the requested wireless simulation inside that world, check whether the physical, link-level, and system-level artifacts are consistent, and improve from the cases where the workflow fails.

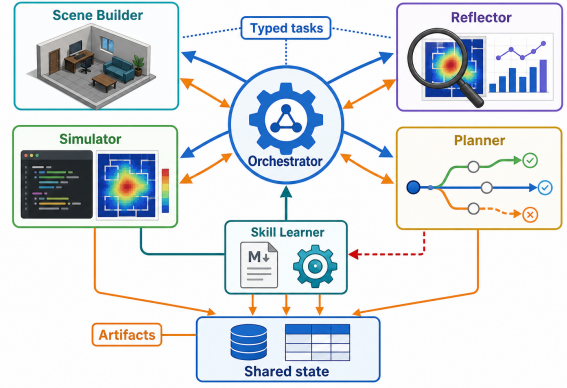


Fig. 3: **Architecture of AGENTICSIONNA.** A central Orchestrator dispatches typed tasks to isolated subagents. The outputs form an executable radio world, bind it to a cross-layer experiment, and feed a skill-learning loop that improves future simulations.

V. SYSTEM DESIGN

AGENTICSIONNA is built around four mechanisms that keep wireless reasoning tied to a coupled radio world and network stack. First, a multi-agent network simulation and optimization architecture specializes the ReAct paradigm [30] into a wireless plan-act-validate-reflect loop over typed artifacts. Second, a shared RAG memory loads learned skills, prior cases, Sionna templates, API notes, and verifier rules without placing the full memory into every worker context. Third, a radio-world construction skill turns natural language into a structured scene coding task whose outputs can be exported to Sionna-compatible XML. Fourth, a skill-improvement loop converts failed trajectories into auditable updates to the procedural skill.

A. Multi-Agent Network Simulation Architecture

ReAct shows that language agents can interleave reasoning with tool actions [30]. Wireless network simulation and optimization need a stricter variant because the intermediate objects are not only text or code snippets: they are radio-world states, Sionna scripts, optimization candidates, plots, numerical metrics, and verifier outcomes whose assumptions must remain consistent across RT, PHY, and system layers. AGENTICSIONNA therefore implements plan-act-validate-reflect as the control logic of a multi-agent architecture rather than as one long monolithic context. In the planning step, the Orchestrator translates the user request into the next typed artifact to produce. In the acting step, a specialized agent edits a scene state, writes Sionna code, runs a simulation, evaluates optimization candidates, or proposes a follow-up experiment. In the validation step, deterministic checks and simulator execution decide whether the artifact is acceptable. In the reflection step, the Reflector explains why validation failed or why a successful run suggests the next experiment or optimization move. For example, validation can flag an invalid material assignment or a missing radio map, while reflection can attribute a coverage gap to wall loss, furniture blockage, distance attenuation, or an inconsistent AP placement assumption.

Agent specialization. Fig. 3 instantiates this loop as a set of specialized agents around a central Orchestrator. The Orchestrator is the entry point and owns global planning: it converts the user request into a task plan containing (1) the scenario and radio-world artifacts, (2) device topology and RF parameters, (3) the simulator layer and task family, (4) required plots, metrics, and result schemas, and (5) validation checks and stopping conditions. The Scene Builder corresponds to the world-construction branch in Fig. 3; it creates or edits the structured radio world, binds architectural geometry to radio materials, and preserves user constraints such as locked objects, device locations, or room dimensions. The Simulator owns the experiment branch; it writes and executes Sionna RT, PHY, and hybrid RT-to-PHY experiments, including parameter sweeps and candidate evaluations for optimization tasks. The Validator sits on the return path from each worker and checks prompt preservation, geometry validity, simulator loadability, script execution, finite outputs, and cross-layer consistency before any artifact is promoted to the shared session state. The Reflector and Experiment Planner implement the reflection branch: the Reflector consumes plots, radio maps, metrics, and scene context to explain failures or coverage trends, while the Planner turns that diagnosis into the next action, such as moving an access point, changing a carrier, tightening a search range, or launching a parameter sweep. Finally, the Summarizer and Skill Learner implement the memory-update branch in Fig. 3 by compressing completed trajectories into session summaries, failure cases, and proposed skill edits. Because these agents communicate through inspectable artifacts rather than free-form transcripts, a scene state, simulation bundle, diagnosis, or skill edit can be validated, replayed, and reused. When the user requests a sweep, the Orchestrator can dispatch multiple Simulator workers in parallel and merge their result schemas.

Shared RAG memory. The agents share an explicit retrieval memory instead of relying on hidden conversational context. This memory contains (1) learned skill blocks, (2) prior successful and failed trajectories, (3) Sionna templates and API notes, (4) verifier rules, and (5) compact session summaries. Retrieval is necessary because this memory is expected to grow: injecting the full skill file, all templates, and all failure notes into every worker would be token-expensive and would distract the worker with procedures for unrelated tasks. The RAG layer therefore acts as the multi-agent memory manager: it loads only the procedural knowledge needed by the current worker and leaves the rest of the memory out of context.

Task-conditioned hierarchical retrieval. Given a user request, the Orchestrator first normalizes it into a structured query with fields for task family, simulator layer, wireless entities, target artifact, scene context, requested metric, and likely verifier constraints. For example, a request such as “generate a 28 GHz coverage map in a built-in urban scenario and compare two base-station positions” activates the RT/coverage family, the selected Sionna scenario, a RadioMapSolver-style template, heatmap and path-gain outputs, and transmitter-placement checks. Retrieval then proceeds coarse-to-fine: (1) a router searches compact module cards and selects one or

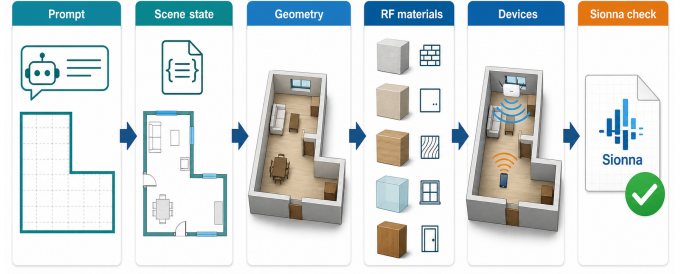


Fig. 4: **Radio-world construction pipeline.** AGENTICSIONNA builds scenes for radio simulation, not only for visual display: geometry is model-first, layout is constraint-checked, materials are radio-aware, devices are explicit, and the final scene is validated by Sionna.

two regions such as radio-world construction, RT coverage, path extraction, RT-to-PHY conversion, BER/BLER simulation, MIMO-OFDM, or system-level mobility; (2) a procedure search runs only inside the selected regions to retrieve concrete skill blocks and worked templates; and (3) a failure/verifier search retrieves narrow notes keyed to the selected procedure, such as invalid Sionna scene names, incompatible antenna-array dimensions, LDPC code-rate limits, resource-grid shape mismatches, or missing radio-map outputs. The Orchestrator then assembles a worker prompt with the user goal, selected skill excerpt, reference templates, expected artifact schema, and verifier checklist. As the skill and failure library grow, token cost scales with the task-specific fragments rather than with the total size of the memory.

B. Radio-World Construction

To simulate a network or optimize a deployment, AGENTICSIONNA must first construct the radio world in which candidate devices, propagation paths, and network decisions are evaluated. We therefore design a dedicated radio-world construction skill rather than treating the environment as a generic 3D generation output. A naive alternative would be to call an existing language-driven 3D scene generator, such as LayoutGPT, FlairGPT, Holodeck, or OptiScene [19]–[21], [36]. These systems are useful for producing visually plausible layouts or embodied-AI environments, but visual plausibility is not the contract required by wireless simulation. For a radio world, correctness depends on whether the scene can expose object geometry, poses, material behavior, and device metadata to the simulator and to later deployment-optimization steps.

Network and radio-environment simulators, with Sionna as one representative backend, typically require a structured scene rather than an opaque generated mesh. Here, a structured scene means a collection of individually addressable 3D mesh objects or geometric primitives, where each object carries metric geometry, coordinate pose, semantic identity, radio-material assignment, and optional device metadata. This object-level structure is what lets the agent independently edit a transmitter, preserve locked furniture during scene revision, assign a high-loss material to one partition without changing the rest of the room, and verify that every RF-relevant object is

covered by a valid material. Naive text-to-3D or image-to-3D generation is therefore insufficient: it may produce an attractive visual artifact, but the result can entangle multiple objects into one surface, hide semantics in texture, omit controllable radio-material mappings, and fail to expose the scene graph that a wireless simulator or verifier needs.

AGENTICSIONNA converts world generation into a constrained scene-programming workflow over a structured scene state. As shown in Fig. 4, the Scene Builder (1) parses the prompt into room geometry, architectural elements, furniture requirements, material hints, device topology, and locked user constraints; (2) derives a hybrid geometric representation for architectural envelopes; (3) grounds furniture in a 3D asset catalog with measured dimensions; (4) assigns object-level radio materials and device metadata; (5) optimizes layout under geometric constraints; and (6) exports and validates the resulting scene through simulator-facing middleware. This pipeline keeps the generated world editable at the object level while still producing the strict simulator artifact needed for ray tracing and downstream network experiments.

Hybrid geometric representation. AGENTICSIONNA does not ask the language model to invent every wall mesh directly. Instead, the Scene Builder emits a compact state with (1) a floor polygon or connected room polygons, (2) room height, (3) wall-segment attributes such as windows and doors, (4) radio-material assignments, (5) furniture categories and selected mesh IDs, and (6) device locations. Deterministic geometry code then derives the simulator geometry from this state. For a rectangular room, the floor polygon contains four vertices; for an L-shaped or multi-room task, it contains the additional concave vertices or connected room polygons. Each adjacent vertex pair becomes a wall segment that is extruded by the room height to form a 3D wall surface, while floors and ceilings are generated from the same polygon. Windows and doors are stored as attributes on wall segments, allowing the geometry code to cut openings and insert glass or door materials while preserving the same wall coordinate system. This representation is hybrid: architectural envelopes are programmatic and consistent, while furniture is selected from the 3D-FUTURE mesh catalog and placed using measured AABBs. The language model therefore controls semantic and RF-relevant choices, such as room shape, material assignment, furniture selection, and placement constraints, while low-level wall geometry is mechanically derived from the floor plan.

Scene middleware. AGENTICSIONNA maintains the radio world as a structured scene state and exports it through multiple middleware formats. A 2D floor plan supports quick human inspection. The Mitsuba representation is the simulator-facing artifact used by Sionna RT. It assigns ITU radio materials to walls, floors, windows, ground, and furniture categories, and reconciles coordinate conventions before Sionna loads the scene. This middleware is important because it gives the agent an editable representation of the world while still producing the strict file format expected by the simulator.

Layout and material validation. The layout stage uses deterministic constraints rather than free-form LLM coordinates.

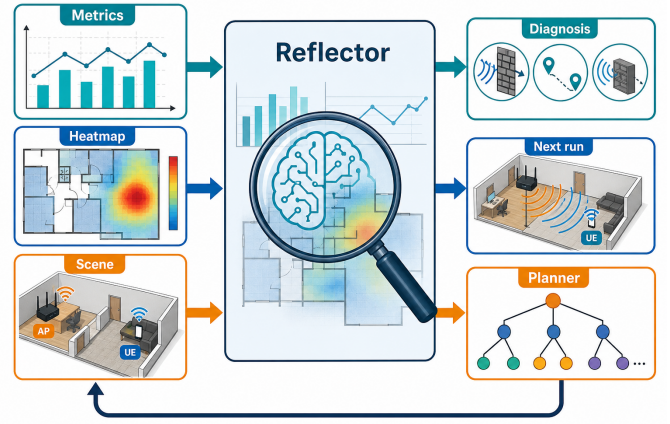


Fig. 5: **Multi-modal reflection.** The Reflector combines numbers, plots, and scene context to produce executable follow-up experiments rather than only natural-language explanations.

The optimizer combines in-bounds placement, AABB collision avoidance, door and pathway clearance, wall affinity, room-shape constraints, and user-specified locks. After export, the verifier checks whether the generated scene preserves the prompt, satisfies geometry and capability constraints, covers architectural and furniture objects with valid radio materials, and loads through Sionna. This final check closes the construction pipeline by ensuring that visual plausibility, object semantics, RF material control, and simulator loadability agree in the same artifact.

C. Simulation Reasoning and Skill Improvement

Once a radio world exists, AGENTICSIONNA must generate the cross-layer wireless experiment that answers the user’s question. The Simulator does not rely on raw documentation alone. It retrieves worked examples for coverage maps, link-quality studies, MIMO experiments, and hybrid RT-to-PHY workflows, then adapts them to the current scene and task. The generated code is screened for valid Sionna imports, channel and coding choices, antenna and MIMO settings, finite outputs, and consistency with the radio world and requested link or system objective. Every successful run emits a result schema containing metrics, plots, simulation parameters, cross-layer assumptions, and provenance.

Correct execution is still not the end of the workflow, because a runnable script can still be wrong for the cross-layer network question. The Reflector consumes numerical metrics, rendered heatmaps, scene context, and user requirements. It diagnoses coverage completeness, signal-strength uniformity, blind-spot location, likely cause, and recommended action. Blind spots are attributed to a closed taxonomy such as wall occlusion, furniture blockage, distance attenuation, interference shadow, or material penetration loss. Fig. 5 illustrates how the Reflector combines spatial plots, metrics, and scene context before proposing follow-up experiments. The Experiment Planner then runs a simulate-reflect-adjust loop and stops when the target is met, confidence is low, or the iteration budget is exhausted.

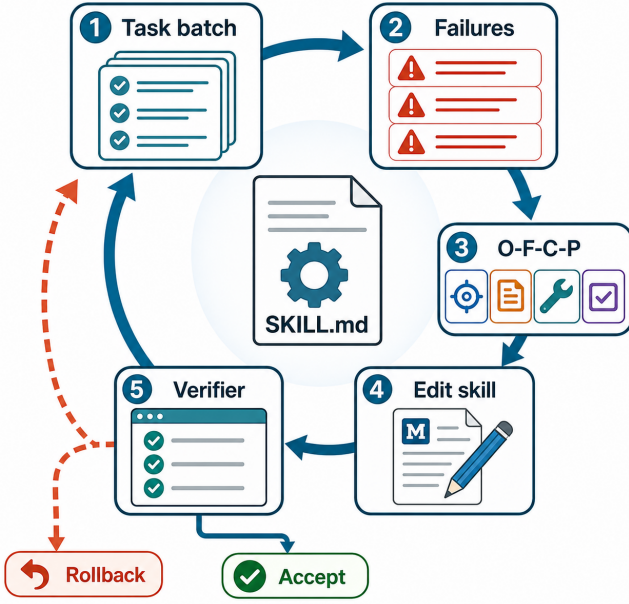


Fig. 6: **Skill improvement as discrete optimization.** AGENTICSIONNA learns from failures by editing its procedural skill, not by changing model weights.

TABLE I: Representative interactive simulation prompt.

Prompt
Create an indoor Sionna dashboard for a furnished room. Build the 3D radio world with walls, furniture, and a TX marker; expose web controls for transmit power, array size, TX position, coverage-grid height/depth/cell size, z-slice, and layer visibility; run the Sionna RT computation on demand; overlay the coverage heatmap and optional ray paths in the 3D viewport; and report BER-vs-SNR, coverage distribution, PDP/OFDM traces, peak signal, mean loss, coverage, frequency, and array statistics so the user can adjust the scenario and recompute.

Failure-driven skill updates. AGENTICSIONNA improves the workflow by editing a human-readable Markdown skill rather than fine-tuning model weights. The update loop has six stages: run a task batch, collect failed trajectories, summarize each failure as an (Observation, Failure, Correction, Principle) tuple, assign the dominant failure to a fixed class, propose one edit to one active skill block, and rerun the verifier before accepting the edit. The failure classes distinguish whether the skill was not consulted, ignored, incomplete, wrong, blocked by model capability, blocked by the environment, or exploited by reward hacking. An edit is accepted only if it improves the train split by at least 2 percentage points and does not regress previously solved tasks. Fig. 6 shows this failure-driven skill update loop.

This loop differs from neural training in three ways. The update unit is a named section, not a gradient step. The rationale is visible in the iteration log. Rollback is a normal version-control operation. These properties are useful in wireless simulation, where users must trust not only that a script ran, but that the assumptions behind the run are inspectable.

VI. IMPLEMENTATION

A. Software Prototype

We implement AGENTICSIONNA as a Python prototype with a multi-agent plan-act-validate-reflect loop adapted from ReAct [30]. The Orchestrator and each worker communicate through typed JSON artifacts rather than free-form chat transcripts: scene workers return a structured scene state, simulator workers return runnable scripts and result schemas, reflectors return typed diagnoses, and the Skill Learner returns a proposed Markdown edit plus a verifier report. The initial task templates and procedural skill are constructed from tutorials spanning the three Sionna layers we exercise in the benchmark: RT, PHY, and SYS. The scene-generation stack uses 3D-FUTURE assets [38], Trimesh for mesh loading and axis-aligned bounding box (AABB) measurement [39], Shapely and SciPy/SLSQP for geometric constraints and layout optimization [40]–[42], and Mitsuba scene descriptions used by Sionna RT [9], [43], [44]. An AABB is the smallest coordinate-aligned box enclosing a mesh; we use its width, depth, and height to test furniture placement, clearance, and collisions without relying on visual inspection. The SciPy/SLSQP constraints encode continuous layout rules such as keeping objects inside the room polygon, maintaining separation between AABBs, leaving door/pathway clearance, and honoring user-specified object locks. The wireless stack wraps high-level Sionna templates for RT, PHY, and SYS simulations and adapts them to the scenario, devices, metrics, and output schema requested by the user. Each run records the prompt, retrieved references, active skill version, generated artifacts, validation outcomes, and simulator logs so that a failed task can be replayed during skill iteration.

B. Model and Hardware Backends

For closed-source model experiments, we call hosted APIs rather than serving model weights locally. The primary backbone in the reported prototype is Claude Sonnet 4.6, and cross-model checks use OpenAI GPT and Anthropic Claude API endpoints [45], [46]. For open-source model experiments, we serve Llama-3.3-70B-AWQ and Qwen3-Coder-30B-A3B-Instruct on NVIDIA A100 GPUs and run the same benchmark harness, validators, and Sionna jobs against the local endpoint. This split keeps the agent code and verifier code fixed while changing only the model backend. Sionna RT, PHY, and SYS experiments execute on GPU-equipped Python environments with TensorFlow and Sionna 2.0 installed [8], [37], [47]; CPU-only checks are used for structural validation, JSON/schema validation, geometry constraints, and artifact existence.

C. Experimental Accounting

We treat skill iteration as discrete program improvement rather than neural training. During development, only the 60-task training split is used to propose and accept skill edits; the 40 held-out test tasks are evaluated once with the skill version, decoding settings, and verifier suite frozen. No held-out failure is used to update the skill, modify the prompt style, or change the category definitions. For every benchmark run,

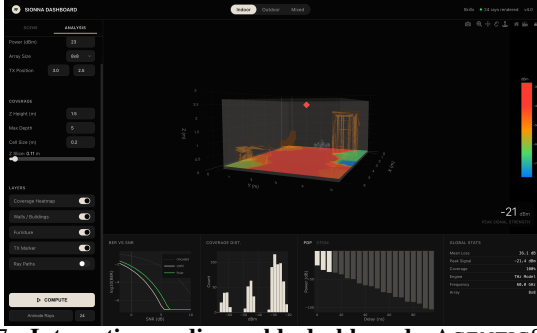


Fig. 7: **Interactive radio-world dashboard.** AGENTICSIONNA exposes generated radio worlds through a web UI that links scene construction, RF controls, simulation execution, and result inspection. Users can adjust configuration as well.

the harness logs input tokens, output tokens, model wall-clock time, verifier wall-clock time, Sionna simulation time, GPU type, and whether the run used an API-hosted or locally served model. These logs support the efficiency and usability analysis reported at the end of the evaluation.

VII. EVALUATION

We evaluate AGENTICSIONNA across three stages of wireless simulation: (1) 3D radio-world generation, (2) ray-tracing and physical-layer simulation, and (3) network/system-level verification. We then include an end-to-end follow-up study that tests whether multi-modal reflection improves coverage over repeated simulations.

A. Experimental Setup

Task corpus. The first stage is a 100-task indoor radio-world suite with 60 train tasks and 40 held-out test tasks. The train split is used only for interactive skill improvement, while the test split is evaluated with the final accepted skill frozen. The 100 tasks are derived from 20 seed scene templates and cover seven capability groups: single-room generation, mixed RF materials, interior partitions, L-shaped geometry, multi-room apartments, building-code-style window constraints, and scene editing. Representative prompts include a 4 m×3 m home office with a desk, chair, and bookshelf; a living room with hardwood, drywall, and glass materials; a drywall-partitioned office; an L-shaped office; a three-bedroom apartment connected by a corridor; an IRC-style bedroom window aperture task; and an edit task that moves a sofa while preserving all other objects and materials.

The second stage uses Sionna built-in scenarios rather than generated scenes so that a failure cannot be attributed to a malformed radio world. It covers four fixed scenarios: two indoor testbeds and two outdoor city scenarios, Outdoor Street Canyon and Outdoor Paris-Etoile in France. The RT+PHY suite has two task types. Simulation tasks include (N1) single-AP simulation with coverage maps or point-level link probes, (N2) AP configuration editing with a before/after rerun, (N3) multi-AP simulation for joint coverage, interference, or serving-AP assignment, and (N4) single-user link simulation for BER/BLER, throughput, SINR, or spectral efficiency.

Optimization tasks stress continuous physical control: (P1) AP azimuth/downtilt optimization for single-user throughput and (P2) beam-angle/TX-power optimization for rate–power trade-offs. These are standard wireless workloads rather than new task definitions: coverage and ray-tracing studies are common in radio planning, link metrics are standard OFDM/NR checks, and beam/power optimization is a classical wireless control problem [2], [8], [9], [48]–[50]. The reported frozen result table covers the completed N1–N2 subset. The third stage evaluates network/system-level simulation with the same two task types. Simulation tasks report system throughput, per-user rates, scheduler behavior, SINR, RB usage, and fairness under a fixed deployment or fixed resource-allocation rule. Optimization tasks change the network configuration, including single-objective multi-cell beamforming for sum capacity and multi-objective RB scheduling for throughput/fairness trade-offs. These system-level tasks follow prior work on multi-cell cooperation and OFDMA resource allocation [51], [52].

Baselines. For radio-world generation, we compare three agent settings. (1) The naive coding agent receives the task context and writes the scene artifacts directly, without our curated radio-world skill. (2) The self-authored skill agent is first asked to write its own procedural skill from the available domain context, and then uses that model-authored skill to generate scenes. (3) AGENTICSIONNA uses our curated skill, which is initialized from tutorials and worked examples and then improved through a bounded number of verifier-driven iterations; in our experiments we stop after at most five accepted skill updates and report the final accepted version in the perversion ablation. For wireless simulation tasks we reuse the same three-condition split (naive coding agent, self-authored skill agent, and AGENTICSIONNA with the curated skill) so that the wireless results can be compared to the radio-world results on equal terms. For iterative experiments, we compare full AGENTICSIONNA against a single-shot executor, a loop without multi-modal reflection, a direct coding agent, and grid search as an upper bound where applicable.

Verification. The benchmark uses a unified verifier with task-specific checks rather than relying only on manual grading. Each task is evaluated at three levels. First, an execution check confirms that the agent produced the required artifacts and that the generated simulation can run. Second, an independent oracle checks correctness: for deterministic simulation tasks we rerun the same scenario with a trusted reference implementation, and for optimization tasks we compute a reference by brute-force grid search or analytical bounds over the allowed action space. The agent is judged against this independently produced result, not against its self-reported explanation. Third, boundary-condition checks enforce domain sanity across dimensions such as geometry validity, power and rate ranges, monotonic trends with SNR or frequency, conservation-style constraints, scheduler dimensionality, and fairness arithmetic. We also manually audit representative pass and fail cases and use that audit to calibrate tolerances, so the automatic verifier is aligned with expert judgment rather than prompt similarity. These checks are intentionally similar to program-verification

benchmarks: they combine artifact presence, independent reference oracles, human spot checks, boundary-condition checks, and domain-specific sanity rules. In summary, generated artifacts must pass structural checks and simulator execution rather than visual inspection alone.

B. Radio Environment Generation Evaluation

Setup. This stage evaluates whether the agent can turn natural language into a simulator-facing 3D radio world. The suite contains 100 tasks over 20 seed scene templates, split into 60 train tasks for skill improvement and 40 held-out test tasks for frozen evaluation. The tasks abstract common scene-generation operations into seven families: basic single-room generation, RF-material variation, partitioned rooms, L-shaped rooms, multi-room apartments, building-code-style window constraints, and locked scene edits. Furniture is selected from the 3D-FUTURE asset catalog, while walls, floors, ceilings, windows, and doors are generated from the structured room geometry described in Section V-B.

Verification. The verifier checks the structured scene state rather than the rendered image alone. It requires valid artifacts, prompt preservation, legal room polygons, in-bounds and collision-free furniture AABBs, valid radio-material assignments, task-specific geometry constraints, and Sionna loadability. For example, an L-shaped task must produce a non-rectangular footprint, a partition task must instantiate the divider with the requested material, a multi-room task must create connected rooms with openings, and a scene-edit task must move only the requested object while preserving locked objects and materials. Fig. 7 shows the user-facing dashboard built around these artifacts. The dashboard is not merely a 3D preview: it lets a user inspect the generated scene, set RF parameters such as transmit power, array size, transmitter location, grid resolution, depth, and z -slice, toggle visible layers such as walls, furniture, heatmaps, transmitter markers, and ray paths, and then run the Sionna-backed computation. The central 3D viewport overlays coverage heatmaps on the constructed world, while the lower panels expose BER/SNR curves, coverage distributions, PDP/OFDM traces, and global statistics. This interface makes the artifact concrete and supports human intervention; the benchmark score still comes from the corresponding structured scene state, exported Sionna scene, and simulation outputs rather than visual appearance alone.

Results. Table II reports the main effect of the learned skill. AGENTICSIONNA reaches 75.0% on the training split and 72.5% on the held-out test split, while both baselines fail to solve any held-out task. The held-out result is important because the test tasks are run only after the skill is frozen; no test failure is used to edit the prompt, modify the skill, or change the verifier. The *Self-Written Skill* baseline also explains why skill learning must be verifier-driven. It asks the model to write its own procedural skill before solving the task, but that skill is not validated. When the self-written skill contains an incorrect rule or an incomplete procedure, it becomes misleading prompt context: the agent treats the generated skill as authoritative and is more likely to follow the

TABLE II: Pass rate on the radio-world generation benchmark.

Condition	Train (60)	Test (40)	Δ test
Without Skill	5.1%	0.0%	—
Self-Written Skill	1.4%	0.0%	0.0 pp
AGENTICSIONNA	75.0%	72.5%	+72.5 pp

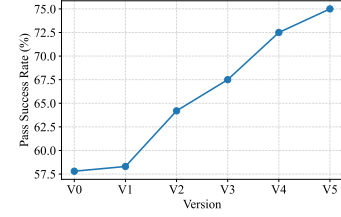


Fig. 8: Self-learning improvement.

wrong procedure. This is why adding an unverified skill can be worse than providing no skill at all on the training split.

The self-learning process on the 60-task training split reveals what the agent learns. As shown in Fig. 8, the accepted skill iterations steadily improve the pass rate from 57.8% at V0 to 75.0% at V5. Early accepted edits add naming and prompt-preservation rules so verifier-visible objects match the user’s request, later iterations compress the capability glossary so relevant rules remain active without consuming the context budget, and the final edit adds a fast path for locked scene edits. The largest gains occur in window/clearance rules, multi-room topology, and partitioned spaces, where the skill translates natural language into executable radio-world requirements that are not explicit in the original prompt.

C. Ray-Tracing and Physical-Layer Simulation

Setup. The second benchmark family asks whether AGENTICSIONNA can write executable Sionna simulations once the radio scenario is already provided. We deliberately use four Sionna built-in scenarios—two indoor testbeds and two outdoor city scenes—so that failures measure simulation coding rather than malformed radio-world construction. The RT+PHY suite has two task types, summarized in Table III. Simulation tasks ask the agent to run the scenario as specified: **N1** is a single-AP simulation that produces a coverage map or a point-level link probe; **N2** edits an AP configuration and reruns the before/after scenario; **N3** deploys multiple APs to evaluate joint coverage, interference, or serving-AP assignment; and **N4** evaluates a single-user link through BER/BLER, throughput, SINR, or spectral efficiency. Optimization tasks ask the agent to choose physical parameters: **P1** controls antenna azimuth and downtilt to maximize single-user OFDM throughput, while **P2** jointly tunes beam angle and TX power to trace a Pareto trade-off between SNR/rate and power consumption. The simulation tasks follow common radio-map, ray-tracing, and OFDM/NR evaluation practice, while the optimization tasks instantiate standard beamforming and power-control objectives [2], [9], [48]–[50]. This design keeps the simple AP-centric tasks, but makes the stage cross-layer: the harder tasks must carry the same AP, UE, channel, and metric assumptions from ray tracing into physical-layer or objective-level computations. Rather than giving the model a long checklist, the benchmark uses short natural-language prompts such as the interactive dashboard

example in Table I; implementation constraints such as output schemas, forbidden shortcuts, and verifier rules are supplied by the retrieved skill and the benchmark harness.

Verification. The verifier combines independent references, human-audited tolerances, and boundary checks. For RT simulation tasks, we generate a trusted reference by rerunning the requested scenario with a fixed implementation and compare the agent’s physical outputs against that reference within tolerance. For PHY link tasks, we compare against analytical baselines when available and otherwise check qualitative laws such as valid ranges and monotonic improvement with SNR. For P1/P2 optimization tasks, we independently sweep the allowed control variables—for example antenna orientation, beam angle, and TX power—and compare the agent’s selected configuration to the brute-force or Pareto-frontier reference. This keeps the score tied to physical behavior and objective quality rather than to whether the agent used exactly the same script structure as the benchmark author.

Runs. Table IV reports the completed frozen subset, N1 and N2, which contains $2 \times 4 \times 3 \times 5 = 120$ trials. Each trial uses Claude Sonnet 4.6 with a 25-turn budget; per-trial wall time for the completed RT subset is about two minutes. The harness logs the prompt, generated script, executed outputs, and structured verifier verdict for every trial.

Results. Table IV reports the pooled pass rate per task and condition for the completed subset. AGENTICSIONNA reaches 95% on N1 and 100% on N2, against 35% for *Without Skill* and 40%/55% for *Self-Written Skill*. The main failure mode in the two baselines is not Python syntax. They often import a closed-form path-loss formula and emit a plausible-looking heatmap without ever loading the scenario or invoking Sionna RT; the self-written skill repeats the same problem because it does not encode the Sionna API conventions for scenario loading, frequency setters, `PlanarArray` antennas, and coverage-map artifacts. AGENTICSIONNA has one N1 failure, caused by a missing output file rather than a physics error, and no N2 failures. The remaining simulation and PHY optimization tasks make this stage stricter than a coverage-map smoke test: they require the agent to preserve the scenario assumptions while moving into multi-AP deployment, UE-level link metrics, and continuous physical control.

D. Network/System-Level Verification

Setup. The third stage evaluates whether the agent can preserve assumptions at the network and system level. The network suite has the same two task types, summarized in Table V. Simulation tasks ask the agent to run the network as specified: one reports overall system throughput and per-user rates for a fixed multi-AP/multi-user deployment, and the other

TABLE III: RT+PHY simulation and optimization task families.

ID	Type	Task
N1	Sim.	Single-AP coverage map or point-level link probe
N2	Sim.	AP configuration edit and before/after rerun
N3	Sim.	Multi-AP coverage, interference, or serving-AP assignment
N4	Sim.	Single-user BER/BLER, throughput, SINR, or spectral efficiency
P1	Opt.	AP azimuth/downtilt $\rightarrow$ maximize one-UE OFDM throughput
P2	Opt.	Beam angle + TX power $\rightarrow$ rate/power Pareto trade-off

TABLE IV: Simulation accuracy.

Task	Without Skill	Self-Written Skill	AGENTICSIONNA
N1 single-AP simulation	35.0%	40.0%	95.0%
N2 AP config. edit	35.0%	55.0%	100.0%

TABLE V: Network simulation and optimization task families.

ID	Type	Task
S1	Sim.	Fixed deployment $\rightarrow$ system throughput and per-user rates
S2	Sim.	Fixed scheduler $\rightarrow$ SINR, RB usage, and fairness trace
S3	Opt.	Multi-cell beamforming $\rightarrow$ maximize sum capacity
S4	Opt.	RB scheduling $\rightarrow$ throughput/fairness trade-off

reports scheduler behavior, SINR, and fairness metrics under a fixed resource-allocation rule. Optimization tasks ask the agent to change the network configuration: the single-objective task tunes multi-cell beamforming vectors to maximize sum capacity under co-channel interference, while the multi-objective task allocates OFDM resource blocks and power across users to balance total throughput against edge-user fairness. These tasks are different from the RT+PHY stage because the desired answer is often a structured system trace or aggregate network metric rather than a single link-quality curve.

Verification. The system-level verifier recomputes key metrics from the returned arrays. For multi-user tasks, it checks that the number of users matches the per-user metric arrays, that all rates are non-negative, and that the reported Jain’s fairness index agrees with the verifier’s recomputation. For throughput and SINR tasks, it checks physical ranges and dimensional consistency. For mobility or handover-style tasks, it checks that each time step has a valid serving cell, that handover events are temporally ordered, and that reported traces are finite.

Results. Table VI summarizes the system-level stage. Final values will be filled from the frozen benchmark log.

Observation. System-level tasks expose a different failure mode than RT or PHY tasks. An agent can write code that executes and even produces plausible aggregate throughput, while silently dropping users, reporting a fairness score inconsistent with the per-user rates, or returning a handover trace with invalid cell identifiers. The verifier therefore treats structured arrays as first-class outputs and recomputes aggregate metrics from them.

E. Follow-Up Simulation and Coverage Improvement

Setup. Although AGENTICSIONNA is framed as a simulation agent rather than only a planner, deployment-style tasks provide a useful stress test: the agent must interpret a result, decide what simulation to run next, and improve the radio world and deployment configuration together. The study uses 27 scenes and gives the agent an initial coverage map plus a target coverage objective.

Verification. The verifier checks the full planning trajectory rather than only the final answer. The agent must emit a planning state with iteration history, coverage values, blind-spot diagnoses, and recommended actions. The history must improve by at least the task threshold, satisfy a near-monotonicity rule, and terminate with a structured reason.

TABLE VI: Network/system-level verification.

Method	Artifact	Exec.	Domain
Without Skill	xx%	xx%	xx%
Self-Written Skill	xx%	xx%	xx%
AGENTICSIONNA	xx%	xx%	xx%

TABLE VII: Ablation Study.

Method	Avg. cov.	Target ach.	Avg. iter.
Grid search	96.2%	100%	N/A
AGENTICSIONNA Full	93.8%	81.5%	3.4
w/o Reflection	88.3%	59.3%	5.1
Single-shot	79.6%	37.0%	1
Raw LLM	41.2%	3.7%	1

Results. Table VII compares full AGENTICSIONNA against ablations.

Observation. Full AGENTICSIONNA reaches within 2.4 percentage points of grid search while using a median of 3.4 iterations. The no-reflection ablation improves more slowly because it lacks spatial cause attribution. This result supports the broader claim that wireless simulation benefits from multi-modal interpretation, not just code execution.

F. Impact of Model

Setup. The main results above use Claude Sonnet 4.6 as the default backbone. To separate the effect of the procedural skill from the effect of the underlying model, we also run cross-model checks while keeping the prompts, tools, verifier, and benchmark harness fixed. Closed-source checks use hosted Claude and OpenAI GPT endpoints, while open-source checks serve the model locally on A100 GPUs.

Results. Table VIII summarizes the open-source model checks. Llama-3.3-70B improves with AGENTICSIONNA, while the Qwen3-Coder-30B partial run regresses under the same harness. This suggests that the procedural skill is not a model-independent patch: weaker or differently tuned models can over-follow retrieved procedures, miss verifier-facing details, or fail to recover from long tool instructions. We therefore report the model backbone separately in all main experiments.

G. Efficiency and Usability

We expect two user groups to benefit differently. Experts use AGENTICSIONNA to reduce boilerplate and run more what-if experiments per unit time. Non-experts use it to start from language rather than from Sionna documentation. We will report final user-facing efficiency numbers after the study is complete: xxx expert speedup, xxx non-expert task completion, and xxx SUS score. Table IX summarizes the resource accounting fields used to report this cost and usability trade-off; final token and timing values will be filled from the frozen experiment logs.

TABLE IX: Token Consumption

Stage	Tokens	Model time	Verifier/sim
3D radio-world generation	in/out	xxx	xxx
Sionna script generation	in/out	xxx	xxx
Reflection/planning	in/out	xxx	xxx
Skill-update proposal	in/out	xxx	xxx

TABLE VIII: Backbone impact.

Model	Without Skill	AGENTICSIONNA	Runs
Llama-3.3-70B	16.9% $\pm$ 1.7	22.2% $\pm$ 2.7	4
Qwen3-Coder-30B	35.8%	23.6%	partial

VIII. CONCLUSION

We presented AGENTICSIONNA, a self-improving LLM agent for wireless network simulation. The system converts natural-language requests into executable radio worlds, runs cross-layer Sionna wireless-simulation experiments, diagnoses multi-modal outputs, and improves its procedural skill from failure traces. By grounding LLM reasoning in 3D radio-environment generation, Sionna worked examples, typed result schemas, and execution-based verifiers, AGENTICSIONNA reduces the expertise barrier around physics-based wireless simulation. The current results show strong gains in radio-world generation and simulation execution; the remaining placeholders will be replaced as the final experiment logs are frozen.

REFERENCES

- [1] T. S. Rappaport, *Wireless Communications: Principles and Practice*, 3rd ed. Pearson, 2024.
- [2] E. Dahlman, S. Parkvall, and J. Sködl, *5G NR: The Next Generation Wireless Access Technology*, 2nd ed. Academic Press, 2020.
- [3] 3GPP, “Study on channel model for frequencies from 0.5 to 100 GHz,” 3GPP TR 38.901, Release 17, 2022.
- [4] ITU-R, “Effects of building materials and structures on radiowave propagation above about 100 MHz,” Recommendation ITU-R P.2040-3, 2023.
- [5] T. R. Henderson, M. Lacage, G. F. Riley, C. Dowell, and J. Kopena, “Network simulations with the ns-3 simulator,” in *Proceedings of the ACM SIGCOMM Conference Demo Session*, 2008.
- [6] Keysight Technologies, “Channel sounding solutions for 5G NR and beyond,” <https://www.keysight.com>, 2024.
- [7] Ranplan Wireless, “Ranplan: Indoor wireless network design and optimization,” <https://www.ranplanwireless.com>, 2024.
- [8] J. Hoydis, S. Cammerer, F. A. Aoudia, A. Vem, N. Binder, G. Marcus, and S. ten Brink, “Sionna: An open-source library for next-generation physical layer research,” *IEEE Communications Magazine*, vol. 61, no. 3, pp. 52–58, 2023.
- [9] J. Hoydis, F. A. Aoudia, S. Cammerer, M. Nimier-David, N. Binder, G. Marcus, and A. Keller, “Sionna RT: Differentiable ray tracing for radio propagation modeling,” *arXiv preprint arXiv:2303.11103*, 2024.
- [10] A. Zubow, Y. Pilz, S. Rösler, and F. Dressler, “Ns3 meets Sionna: Using realistic channels in network simulation,” *arXiv preprint arXiv:2412.20524*, 2024.
- [11] T. Iye, M. Sakamoto, S. Takaya, E. Sato, Y. Susukida, Y. Nagaoka, K. Maruta, and J. Nakazato, “Open wireless digital twin: End-to-end 5G mobility emulation in O-RAN framework,” *arXiv preprint arXiv:2503.12177*, 2025.
- [12] J. Yang, C. E. Jimenez, A. Wettig, K. Liber, K. Narasimhan, and O. Press, “SWE-agent: Agent-computer interfaces enable automated software engineering,” *arXiv preprint arXiv:2405.15793*, 2024.
- [13] X. Wang, B. Chen, Z. Li *et al.*, “OpenDevin: An open platform for AI software developers as generalist agents,” *arXiv preprint arXiv:2407.16741*, 2024.
- [14] J. Tong, Z. Li, F. Liu, W. Guo, and J. Zhang, “WirelessAgent: Large language model agents for intelligent wireless networks,” *arXiv preprint arXiv:2505.01074*, 2025.
- [15] —, “WirelessAgent++: Automated agentic workflow design and benchmarking for wireless networks,” *arXiv preprint arXiv:2603.00501*, 2026.
- [16] Z. Wang *et al.*, “Large language model agents for radio map generation and wireless network planning,” *arXiv preprint arXiv:2501.11283*, 2025.
- [17] Y. Wang, Y. Zhang, and W. Chen, “BSS-LLM: Large language model as catalyst for base station siting,” *arXiv preprint arXiv:2408.03631*, 2024.

- [18] H. Li, M. Xiao, K. Wang, R. Schober, D. I. Kim, and Y. L. Guan, "ComAgent: Multi-LLM based agentic AI empowered intelligent wireless networks," *arXiv preprint arXiv:2601.19607*, 2026.
- [19] W. Feng, W. Zhu, T.-J. Fu, V. Jha, A. Calber, M. Chang, and W. Y. Wang, "LayoutGPT: Compositional visual planning and generation with large language models," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [20] Y. Yang, F.-Y. Sun, L. Weihs, E. VanderBilt, A. Herrasti, W. Han, J. Wu, N. Haber, R. Krishna, L. Liu *et al.*, "Holodeck: Language guided generation of 3D embodied AI environments," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [21] Anonymous, "OptiScene: LLM-driven indoor scene layout generation via scaled human-aligned data synthesis and multi-stage preference optimization," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [22] S. Lin, J. Zhou, and M. Yu, "An LLM-based agentic framework for accessible network control," in *AI Crossroads Workshop at ACM SIGMETRICS*, 2025.
- [23] S. Hussain and C. Brennan, "RadioSim Agent: Combining large language models and deterministic EM simulators for interactive radio map analysis," *arXiv preprint arXiv:2511.05912*, 2025.
- [24] Anonymous, "Toward generative 6G simulation: An experimental multi-agent LLM and ns-3 integration," *arXiv preprint arXiv:2503.13402*, 2025.
- [25] —, "Vision-language-model-guided differentiable ray tracing for fast and accurate multi-material RF parameter estimation," *arXiv preprint arXiv:2601.18242*, 2026.
- [26] Y. Qi *et al.*, "FeaGPT: An end-to-end agentic-AI for finite element analysis," *arXiv preprint arXiv:2510.21993*, 2025.
- [27] Anonymous, "AutoMOOSE: An agentic AI for autonomous phase-field simulation," *arXiv preprint arXiv:2603.20986*, 2026.
- [28] —, "MCP-SIM: A self-correcting multi-agent LLM framework for language-based physics simulation," *npj Artificial Intelligence*, 2025.
- [29] —, "COSMO-Agent: Tool-augmented agent for closed-loop optimization, simulation, and modeling orchestration," *arXiv preprint arXiv:2604.05547*, 2026.
- [30] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.
- [31] R. Xu and Y. Yan, "Agent skills for large language models: Architecture, acquisition, security, and the path forward," *arXiv preprint arXiv:2602.12430*, 2026.
- [32] Anonymous, "SoK: Agentic skills—beyond tool use in LLM agents," *arXiv preprint arXiv:2602.20867*, 2026.
- [33] X. Zhang *et al.*, "AFlow: Automating agentic workflow generation," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.
- [34] P. Xia, J. Chen, H. Wang, J. Liu *et al.*, "SKILLRL: Evolving agents via recursive skill-augmented reinforcement learning," *arXiv preprint arXiv:2602.08234*, 2026.
- [35] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" *arXiv preprint arXiv:2310.06770*, 2024.
- [36] X. Zhou, S. Qian, and W. Feng, "FlairGPT: Repurposing LLMs for interior designs," *arXiv preprint arXiv:2404.01067*, 2024.
- [37] NVIDIA, "Sionna documentation," <https://nvlabs.github.io/sionna/>, 2026, accessed 2026-05-19.
- [38] H. Fu, R. Jia, L. Gao, M. Gong, B. Zhao, S. Maybank, and D. Tao, "3D-FUTURE: 3D furniture shape with texture," *International Journal of Computer Vision*, vol. 129, pp. 3313–3337, 2021.
- [39] Dawson-Haggerty *et al.*, "Trimesh: Python library for loading and using triangular meshes," <https://trimesh.org/>, 2024.
- [40] Shapely Contributors, "Shapely: Manipulation and analysis of geometric objects in the Cartesian plane," <https://shapely.readthedocs.io>, 2024.
- [41] P. Virtanen, R. Gommers, T. E. Oliphant *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.
- [42] D. Kraft, "A software package for sequential quadratic programming," *DFVLR Forschungsbericht*, vol. 88, no. 28, 1988.
- [43] M. Nimier-David, D. Vicini, T. Zeltner, and W. Jakob, "Mitsuba 2: A retargetable forward and inverse renderer," *ACM Transactions on Graphics*, vol. 38, no. 6, pp. 1–17, 2019.
- [44] W. Jakob, S. Speierer, N. Roussel, M. Nimier-David, D. Vicini, T. Zeltner, B. Nicolet, K. Camsari *et al.*, "Mitsuba 3 renderer," *ACM Transactions on Graphics*, 2022.
- [45] OpenAI, "GPT-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
- [46] Anthropic, "The Claude model family," *Technical Report*, 2024, <https://www.anthropic.com>.
- [47] M. Abadi, P. Barham, J. Chen *et al.*, "TensorFlow: Large-scale machine learning on heterogeneous systems," <https://www.tensorflow.org/>, 2015.
- [48] Z. Yun and M. F. Iskander, "Ray tracing for radio propagation modeling: Principles and applications," *IEEE Access*, vol. 3, pp. 1089–1100, 2015.
- [49] M. Chiang, C. W. Tan, D. P. Palomar, D. O'Neill, and D. Julian, "Power control by geometric programming," *IEEE Transactions on Wireless Communications*, vol. 6, no. 7, pp. 2640–2650, 2007.
- [50] Q. Shi, M. Razaviyayn, Z.-Q. Luo, and C. He, "An iteratively weighted MMSE approach to distributed sum-utility maximization for a MIMO interfering broadcast channel," *IEEE Transactions on Signal Processing*, vol. 59, no. 9, pp. 4331–4340, 2011.
- [51] D. Gesbert, S. Hanly, H. Huang, S. Shamai Shitz, O. Simeone, and W. Yu, "Multi-cell MIMO cooperative networks: A new look at interference," *IEEE Journal on Selected Areas in Communications*, vol. 28, no. 9, pp. 1380–1408, 2010.
- [52] F. Shams, G. Bacci, and M. Luise, "A survey on resource allocation techniques in OFDM(A) networks," *Computer Networks*, vol. 65, pp. 129–150, 2014.